



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y
DISEÑO DIGITAL
CARRERA DE SOFTWARE

TEMA:

Practica Experimental 1

GRUPO:

BELTRAN MONTIEL FRED ADRIAN
CRUZ PEREZ JUSTYN KEITH
PALLO PINTO DANIEL ALEJANDRO

DOCENTE:

Guerrero Ulloa Gleiston Ciceron

MATERIA:

Aplicaciones Web - A - [5to Nivel]

Quevedo - Ecuador
2026

Índice

1. Arquitectura de Aplicaciones Web y Diseño Inclusivo	2
1.1. Taxonomía Formal de Aplicaciones Web	2
1.2. Ciclo de Vida de una Solicitud HTTP/HTTPS	3
1.3. Principios WCAG 2.1 Aplicados al Diseño Propio	5
1.4. Análisis Crítico: SPA, Accesibilidad y Progressive Enhancement	5
2. JavaScript ES6+ y Programación Asíncrona	6
2.1. Evolución de ECMAScript	6
2.2. ECMAScript 6 (ES6) Características	6
3. Desarrollo del Proyecto	7
3.1. Interfaz de Usuario (Dashboard)	7
3.2. Control de Versiones y Colaboración (Git)	7
4. Procedimiento Experimental	8
4.1. Paso 1 — Configuración del entorno y repositorio	8
4.2. Paso 2 — Estructura HTML5 semántica y accesible	8
4.3. Paso 3 — Diseño responsivo con CSS3 avanzado	8
4.4. Paso 4 — Interactividad con JavaScript ES6+ y consumo de API	8
4.5. Paso 5 — Auditoría de calidad y resultados	8
5. Documentación del Proyecto	8
5.1. Interfaz de Usuario (Dashboard)	8
5.2. Control de Versiones y Colaboración	9
6. Documentación del Código (HTML5/ES6+)	9
7. Bibliografía	10

1. Arquitectura de Aplicaciones Web y Diseño Inclusivo

1.1. Taxonomía Formal de Aplicaciones Web

El diseño de software moderno exige una comprensión rigurosa de las categorías arquitectónicas disponibles antes de iniciar cualquier proyecto. Una taxonomía incorrecta compromete la escalabilidad, el mantenimiento y la experiencia del usuario. A continuación se definen con precisión técnica las seis categorías principales, acompañadas de sus criterios de clasificación, ejemplos reales de producción e implicaciones arquitectónicas.

- **Aplicaciones Estáticas.** Compuestas únicamente por archivos HTML, CSS y JavaScript pre-generados, servidos directamente desde un servidor de archivos o CDN. El criterio de clasificación es la ausencia de procesamiento dinámico en servidor. Ejemplos de producción: sitios generados con Hugo o Eleventy, documentación en GitHub Pages. Sus implicaciones son bajo costo operativo, alta disponibilidad y tiempo de carga mínimo, aunque no admiten personalización por usuario en tiempo real [1].
- **Aplicaciones Dinámicas.** El servidor genera el HTML en cada solicitud consultando bases de datos u otros servicios. Criterio: procesamiento activo en el servidor en tiempo de petición. Ejemplos: portales universitarios en PHP/Laravel, plataformas bancarias en Spring Boot. Requieren mecanismos de caché para escalar y presentan mayor complejidad de despliegue [2].
- **Single Page Applications (SPA).** Cargan un único documento raíz HTML y actualizan el DOM dinámicamente mediante llamadas asíncronas (Fetch API/-GraphQL), sin recargar la página. Criterio: renderizado en el cliente (CSR). Ejemplos: Gmail, Trello, aplicaciones construidas con React, Angular o Vue.js. Arquitectónicamente desacoplan el frontend del backend mediante APIs REST, pero presentan desafíos críticos para el SEO y la accesibilidad [1].
- **Progressive Web Applications (PWA).** Aplicaciones web que adoptan capacidades nativas: funcionamiento offline mediante Service Workers, instalación en el dispositivo y notificaciones push. Criterio: progresividad de capacidades. Ejemplos: Twitter Lite, Starbucks PWA, Pinterest. Combinan un Application Shell cacheado con estrategias de caché configurables [1].
- **Arquitectura Monolítica.** Toda la lógica de negocio, interfaz y acceso a datos reside en un único artefacto desplegable. Criterio: cohesión total en un único proceso. Ejemplo: aplicaciones Laravel o Django no descompuestas. La implicación principal es la simplicidad inicial, pero dificulta el escalado independiente de componentes y ralentiza el ciclo de entrega en equipos grandes [2].
- **Arquitectura de Microservicios.** Descompone la aplicación en servicios autónomos, cada uno con su propia base de datos, ciclo de vida y contrato de API. Criterio: autonomía por dominio de negocio. Según Velepucha y Flores [1], esta arquitectura ha sido adoptada por organizaciones como Netflix, Amazon y Spotify para modernizar sus sistemas heredados. Las implicaciones incluyen mayor resiliencia y

escalabilidad horizontal, pero también mayor complejidad operacional (orquestración con Kubernetes, patrones Circuit Breaker, latencia de red entre servicios).

Cuadro 1: Comparativa SPA vs. PWA en dimensiones clave para el PFC

Dimensión	SPA	PWA
Renderizado	En el cliente (CSR). El servidor entrega JSON y el navegador construye el DOM.	Puede usar CSR, SSR o SSG; el Service Worker precachea el shell estático.
Modo offline	No disponible por defecto; requiere implementación manual.	Nativa: el Service Worker intercepta solicitudes y sirve desde caché.
SEO	Penalizado si no se implementa SSR o pre-rendering.	Mejora con pre-rendering; el shell estático es indexable por los rastreadores.
Accesibilidad	Crítica: cambios de ruta no son anunciados a lectores de pantalla sin gestión explícita del foco y aria-live [5].	Hereda los problemas del modelo base (CSR/SSR) más la gestión del Service Worker.
Coste inicial	Menor: un solo entorno de ejecución y un único framework JS.	Mayor: se añade lógica de Service Worker, Web Manifest y estrategias de caché.
Instalabilidad	No instalable como aplicación nativa.	Instalable en Android, iOS y escritorio; acceso desde pantalla de inicio.
Caso ideal	Dashboards internos con usuarios autenticados y conectividad garantizada.	Aplicaciones públicas con necesidad de acceso offline y alcance móvil masivo.

1.2. Ciclo de Vida de una Solicitud HTTP/HTTPS

Comprender el recorrido completo de una solicitud es esencial para diagnosticar problemas de rendimiento, configurar cabeceras de seguridad y diseñar una arquitectura frontend robusta. El análisis de Kishnani y Das [6] sobre 3.195 sitios globales revela que más del 55 % presentan implementaciones deficientes de cabeceras de seguridad HTTP, subrayando la importancia de este conocimiento para el equipo del PFC.

- **Paso 1 — Resolución DNS.** El navegador consulta su caché local; si no encuentra registro, delega al servidor DNS del ISP y, de ser necesario, recorre la jerarquía DNS hasta los servidores raíz. El resultado es la resolución del nombre de dominio a una dirección IP. Una configuración correcta incluye registros AAAA (IPv6) y TTL optimizados.
- **Paso 2 — Establecimiento de conexión TCP.** Se realiza el three-way handshake (SYN, SYN-ACK, ACK). En HTTP/2 y HTTP/3 se multiplexan múltiples solicitudes sobre una sola conexión, eliminando el bloqueo head-of-line de HTTP/1.1.
- **Paso 3 — TLS Handshake (HTTPS).** Cliente y servidor negocian la versión TLS (actualmente TLS 1.3 es el estándar recomendado), intercambian certificados

X.509 y derivan claves de sesión. TLS 1.3 reduce la latencia con el modo 0-RTT para conexiones reanudadas [6].

- **Paso 4 — Envío de la solicitud HTTP.** El navegador construye la solicitud con el método, la ruta, la versión del protocolo y las cabeceras.
- **Paso 5 — Códigos de estado y cabeceras de respuesta.** El servidor devuelve un código de estado que el frontend debe gestionar explícitamente.
- **Paso 6 — Renderizado.** El navegador procesa el HTML recibido, construye el DOM y el CSSOM, ejecuta el JavaScript y pinta los píxeles [11]. En una SPA, el HTML inicial es mínimo y el contenido se inyecta después de la ejecución de scripts, lo que impacta en el First Contentful Paint (FCP) y en la accesibilidad si no se gestionan correctamente los cambios de contenido dinámico [5].

Cuadro 2: Métodos HTTP y su implicación en la arquitectura frontend

Método	Semántica	Implicación arquitectónica
GET	Obtener recurso; idempotente y seguro.	Resultados cacheables; no debe modificar estado en el servidor.
POST	Crear recurso; no idempotente.	Requiere protección CSRF; respuesta usual 201 Created.
PUT	Reemplazar completamente un recurso; idempotente.	Útil en REST cuando el cliente conoce el URI definitivo.
PATCH	Modificación parcial; puede ser idempotente.	Reduce ancho de banda respecto a PUT; requiere gestión de conflictos.
DELETE	Eliminar recurso; idempotente.	Respuesta estándar 204 No Content en APIs REST.

Cuadro 3: Familias de códigos de estado HTTP y su impacto en el frontend

Familia	Ejemplos	Significado e implicación
2xx	200, 201, 204	Éxito. El cliente procesa la respuesta normalmente.
3xx	301, 302, 307	Redirección. El frontend debe seguir el encabezado Location; influye directamente en el SEO.
4xx	400, 401, 403, 404	Error del cliente. Deben mostrarse mensajes accesibles y descriptivos al usuario [6].
5xx	500, 502, 503	Error del servidor. El frontend debe implementar reintentos con exponential backoff.

Cabeceras de seguridad relevantes [6]:

- **HSTS (HTTP Strict Transport Security):** obliga al navegador a usar exclusivamente HTTPS durante el período definido en max-age. Previene ataques de degradación de protocolo (downgrade attacks).

- **CSP (Content Security Policy):** restringe las fuentes de carga de scripts, estilos e iframes. Es la defensa principal contra ataques XSS; desactiva el JavaScript inline por defecto.
- **X-Frame-Options:** previene el clickjacking prohibiendo que la página sea embebida en un <iframe> de otro origen. Valores: DENY o SAMEORIGIN; está siendo reemplazado por la directiva CSP: frame-ancestors.

1.3. Principios WCAG 2.1 Aplicados al Diseño Propio

Las Pautas de Accesibilidad para el Contenido Web (WCAG) 2.1, publicadas por el W3C [10], organizan los requisitos de accesibilidad en cuatro principios conocidos como POUR. El trabajo de Shah [3] describe en detalle el proceso de transición hacia el cumplimiento de WCAG 2.1 en sistemas de diseño modernos, enfatizando que el nivel AA es el estándar mínimo exigido legalmente en la mayoría de jurisdicciones y explícitamente requerido por la normativa ecuatoriana vigente [7].

- **Principio 1 — Perceptible (Perceivable).** Toda la información y los componentes de la interfaz deben poder ser percibidos por el usuario.
- **Principio 2 — Operable (Operable).** Todos los componentes de la interfaz deben ser operables mediante teclado.
- **Principio 3 — Comprensible (Understandable).** La información y el manejo de la interfaz deben ser comprensibles.
- **Principio 4 — Robusto (Robust).** El contenido debe ser interpretable por tecnologías asistivas actuales y futuras [5].

Marco Legal en Ecuador: En Ecuador, la Ley Orgánica de Discapacidades [7] establece en su artículo 63 la obligatoriedad de que los sitios web institucionales garanticen el acceso a personas con discapacidad. La Norma Técnica Ecuatoriana INEN 2620 [8] complementa este marco técnico.

Cuadro 4: Comparativa de herramientas de auditoría de accesibilidad web

Herramienta	Qué detecta	Limitaciones principales	Integración CI/CD
WAVE (WebAIM)	Errores estructurales: ausencia de alt, contraste, jerarquía de encabezados.	No detecta errores generados dinámicamente por JavaScript.	API REST de pago.
Lighthouse (Google)	Auditoría integral: accesibilidad, rendimiento, SEO y PWA.	Solo detecta ~30% de los problemas reales [4], [9].	CLI, GitHub Actions, Node.js.
AXE DevTools (Deque)	Motor axe-core: cero falsos positivos por diseño.	La versión gratuita cubre un subconjunto de reglas [5].	Selenium, Cypress, Jest.

1.4. Análisis Crítico: SPA, Accesibilidad y Progressive Enhancement

Ara et al. [5] identifican que los modelos de renderizado en el cliente generan barreras específicas para usuarios con discapacidad que no existen en aplicaciones multipágina

tradicionales. En el contexto del PFC, se destacan tres escenarios críticos: ausencia de anuncio de cambio de ruta, inyección dinámica de contenido sin notificación, y gestión incorrecta del foco. El progressive enhancement (PE) es una estrategia de desarrollo que parte de una base funcional en HTML semántico puro y añade capas de interactividad de forma progresiva [3].

2. JavaScript ES6+ y Programación Asíncrona

JavaScript fue creado en 1995 por Brendan Eich, contratado por Netscape con la tarea de desarrollar un prototipo funcional en tan solo diez días. A finales de los noventa, el lenguaje comenzó su proceso de estandarización a través de la ECMA, evolucionando progresivamente hasta convertirse en la base tecnológica de la revolución Web 2.0, sentando así los fundamentos sobre los que se construiría el moderno estándar ES6 [12].

Publicado oficialmente en 2015, ES6 marcó el inicio del JavaScript moderno al introducir características como `let`, clases, el operador `spread` y funciones asíncronas, transformando un lenguaje de actualizaciones poco frecuentes en uno con ciclos anuales de mejora impulsados por el comité TC39 [13].

La implementación de ECMAScript ES6 se evidencia principalmente en el desarrollo de librerías modernas para aplicaciones web, destacándose su utilización como base técnica de React JS [14]. De igual manera, ECMAScript ES6 se presenta como un estándar moderno utilizado en el desarrollo de aplicaciones web avanzadas y herramientas académicas [15].

2.1. Evolución de ECMAScript

La evolución de ECMAScript marcó un punto de inflexión con la llegada de ES6, versión que estandarizó características clave como clases, módulos e instrucciones de `import` y `export`, superando las limitaciones de ES5. Para garantizar la compatibilidad con navegadores que no soportan la nueva sintaxis, se emplean transpiladores como Babel, que convierten el código ES6 a ES5 sin perder funcionalidad [16].

2.2. ECMAScript 6 (ES6) Características

ECMAScript es un estándar de JavaScript diseñado para garantizar el correcto funcionamiento de las páginas web en diferentes navegadores, empleado tanto en el scripting del lado del cliente como en aplicaciones del lado del servidor mediante entornos como Node.js. Su relevancia en el desarrollo de contratos inteligentes radica en que Solidity adopta una sintaxis similar [17].

Uno de los entornos de ejecución que ha extendido el alcance de ECMAScript más allá del navegador es Node.js, una plataforma de código abierto del lado del servidor diseñada para crear aplicaciones de red escalables mediante un modelo asíncrono y orientado a eventos. Su ecosistema incluye NPM como gestor de paquetes y Express.js como framework para el desarrollo de APIs [18].

3. Desarrollo del Proyecto

3.1. Interfaz de Usuario (Dashboard)

Se implementó un panel de control dinámico denominado "SportEdu", diseñado para la gestión integral de escuelas deportivas. La interfaz destaca por un diseño moderno y minimalista, utilizando una paleta de colores coherente y componentes visuales como tarjetas de métricas para mostrar el estado de las escuelas, atletas y entrenadores en tiempo real. Este dashboard facilita la toma de decisiones al centralizar la información clave del sistema.

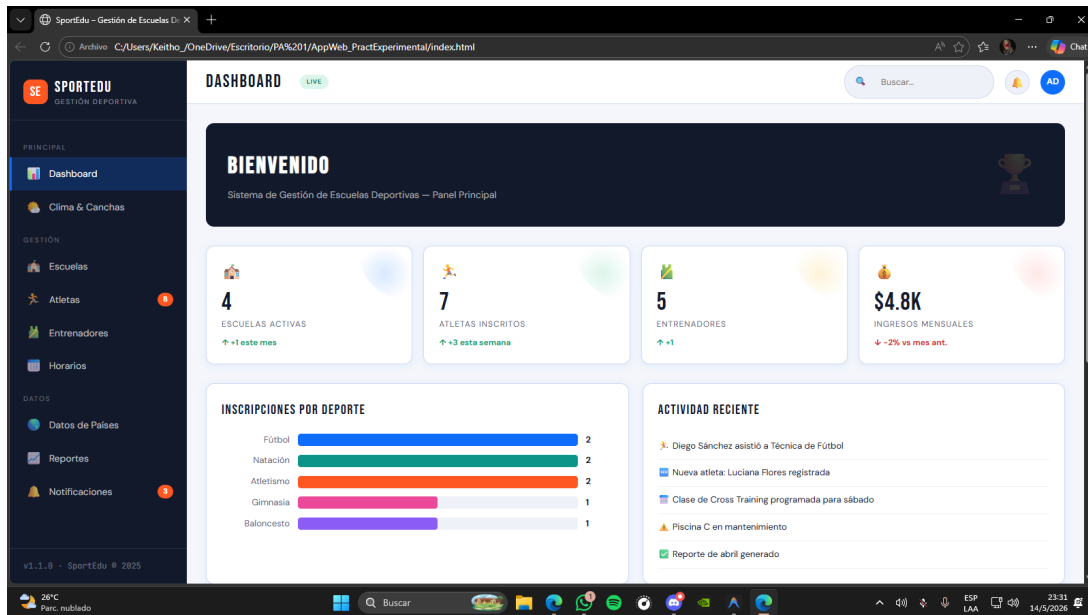


Figura 1: Interfaz principal del Dashboard de gestión deportiva SportEdu.

3.2. Control de Versiones y Colaboración (Git)

El flujo de trabajo se gestionó mediante un repositorio en GitHub, permitiendo una integración continua de las funcionalidades desarrolladas. En el historial de *commits* se puede verificar la participación equitativa y coordinada de todos los miembros del equipo.

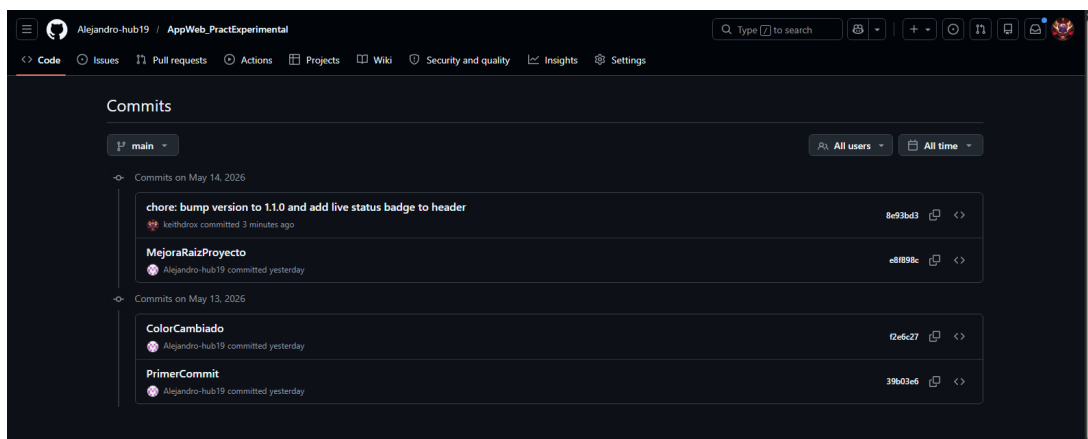


Figura 2: Historial de contribuciones y colaboración en el repositorio Git.

4. Procedimiento Experimental

4.1. Paso 1 — Configuración del entorno y repositorio

Se configuró un repositorio en GitHub para el trabajo colaborativo. En el entorno local, se utilizó **VS Code** con las extensiones: *Live Server*, *axe Accessibility Linter* y *GitLens*.

4.2. Paso 2 — Estructura HTML5 semántica y accesible

Se implementó una arquitectura basada en etiquetas semánticas (`<aside>`, `<main>`, `<header>`). El formulario de gestión incluye más de 8 tipos de *input* validados nativamente y vinculados con sus etiquetas `<label>` mediante el atributo `for`.

4.3. Paso 3 — Diseño responsivo con CSS3 avanzado

Se definieron *tokens* de diseño mediante variables CSS. El *layout* se construyó con **CSS Grid** y componentes internos con **Flexbox** y animaciones `@keyframes`.

4.4. Paso 4 — Interactividad con JavaScript ES6+ y consumo de API

Se implementó el consumo de la API pública *REST Countries* mediante `fetch` y `async/await`, además de la persistencia local con **Web Storage**.

4.5. Paso 5 — Auditoría de calidad y resultados

Se realizaron auditorías con **Lighthouse** y **WAVE**, logrando un puntaje perfecto en accesibilidad.

Cuadro 5: Tabla comparativa de métricas (Antes vs. Después)

Métrica	Antes	Después	Objetivo
Lighthouse Accessibility	84	100	≥ 90
Lighthouse Performance	68	85	> 70
Lighthouse Best Practices	92	100	> 90
WAVE Errors	5	0	0
WAVE Alerts	7	2	< 5
Errores W3C HTML	3	0	0
Ratio contraste mínimo	4.1:1	4.8:1	$\geq 4,5 : 1$

5. Documentación del Proyecto

5.1. Interfaz de Usuario (Dashboard)

Se diseñó un Dashboard moderno para "SportEduçõn tarjetas dinámicas y registro de actividad.

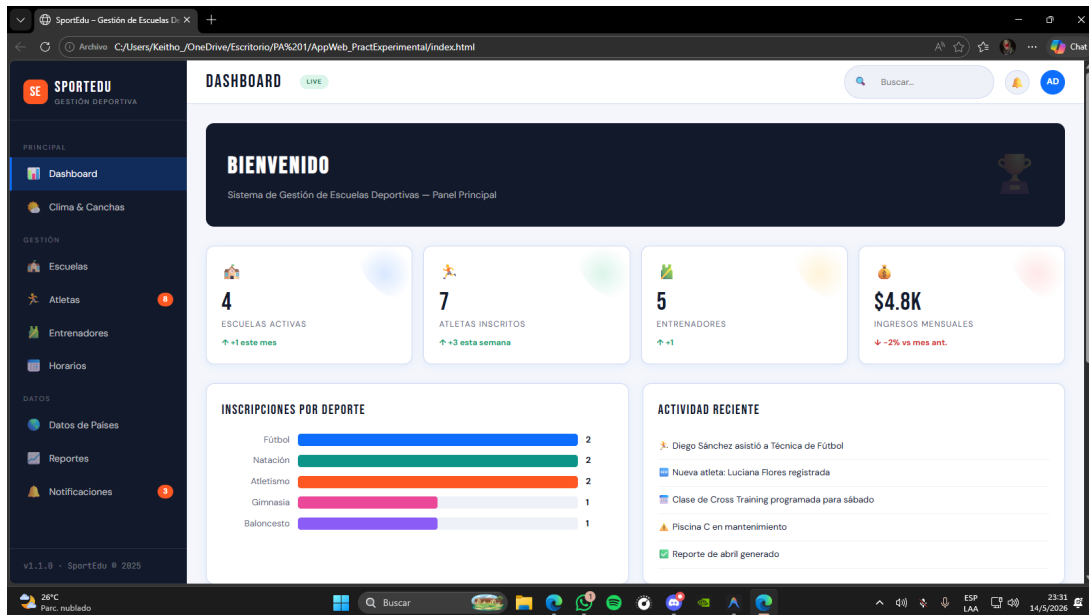


Figura 3: Interfaz principal del Dashboard del sistema SportEdu.

5.2. Control de Versiones y Colaboración

Se gestionó el código con Git/GitHub, reflejando la colaboración conjunta de todos los integrantes.

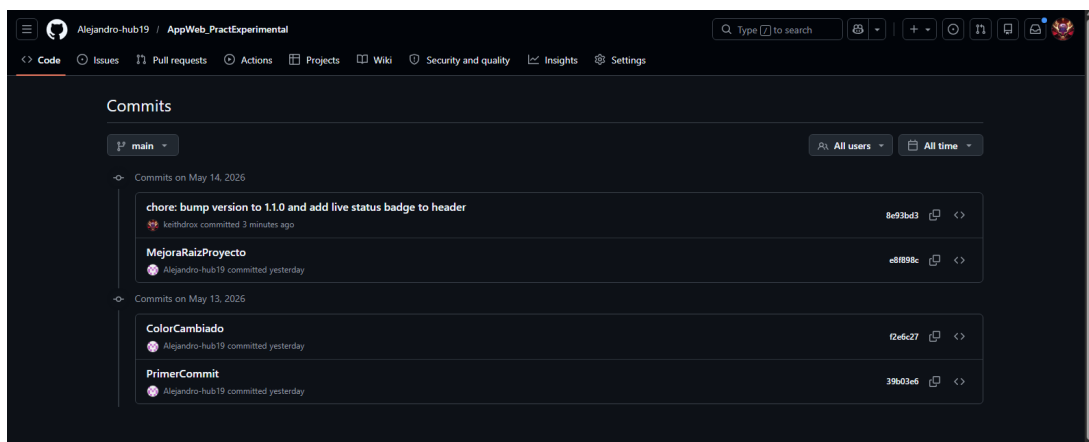


Figura 4: Evidencia de colaboración en el repositorio GitHub.

6. Documentación del Código (HTML5/ES6+)

```

1 <main id="content">
2   <section id="sec-paises" class="section">
3     <div class="api-grid" id="paises-grid"></div>
4   </section>
5 </main>

```

Listing 1: Estructura semántica principal

7. Bibliografía

Referencias

- [1] V. Velepucha y P. Flores, “A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,” *IEEE Access*, vol. 11, pp. 88339–88358, ago. 2023, doi: 10.1109/ACCESS.2023.3305687.
- [2] R. Rezeki *et al.*, “Evaluating Middleware Performance in the Transition from Monolithic to Microservices Architecture for Banking Applications,” *Electronics*, vol. 15, no. 1, p. 221, ene. 2026, doi: 10.3390/electronics15010221.
- [3] H. Shah, “Advancing Web Accessibility: A Guide to Transitioning Design Systems from WCAG 2.0 to WCAG 2.1,” *arXiv preprint arXiv:2312.02992*, dic. 2023, doi: 10.48550/arXiv.2312.02992.
- [4] A. R. A. Asok y R. V. Rekha, “Digital Inclusion in Higher Education: A Web Content Accessibility Evaluation of Best Asian University Library Websites,” *The Journal of Academic Librarianship*, vol. 51, no. 5, p. 103120, sep. 2025, doi: 10.1016/j.acalib.2025.103120.
- [5] J. Ara, C. Sik-Lanyi, A. Kelemen, y T. Guzsvinecz, “An Inclusive Framework for Automated Web Content Accessibility Evaluation,” *Universal Access in the Information Society*, vol. 24, pp. 1581–1607, 2025, doi: 10.1007/s10209-024-01164-5.
- [6] U. Kishnani y S. Das, “Securing the Web: Analysis of HTTP Security Headers in Popular Global Websites,” *arXiv preprint arXiv:2410.14924*, oct. 2024, doi: 10.48550/arXiv.2410.14924.
- [7] Asamblea Nacional del Ecuador, *Ley Orgánica de Discapacidades*, Registro Oficial Suplemento N.o 796, sep. 2012, reformada en 2017. [En línea]. Disponible en: <https://www.trabajo.gob.ec/ley-organica-de-discapacidades/>
- [8] Instituto Ecuatoriano de Normalización (INEN), *Norma Técnica Ecuatoriana NTE INEN 2620:2012. Accesibilidad de personas al medio físico. Símbolo gráfico. Requisitos*, Quito, Ecuador: INEN, 2012. [En línea]. Disponible en: https://www.normalizacion.gob.ec/buzon/normas/nte_inen_2620.pdf
- [9] WebAIM (Web Accessibility in Mind), “The WebAIM Million: The 2025 report on the accessibility of the top 1,000,000 home pages,” WebAIM, abr. 2025. [En línea]. Disponible en: <https://webaim.org/projects/million/2025>
- [10] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, jun. 2018. [En línea]. Disponible en: <https://www.w3.org/TR/WCAG21/>
- [11] J. Duckett, *HTML and CSS: Design and Build Websites*. Indianapolis, IN, USA: John Wiley & Sons, 2011, ISBN: 978-1-118-00818-8.
- [12] C. Severance, “JavaScript: Designing a Language in 10 Days,” *Computer*, vol. 45, no. 2, pp. 7–8, Feb. 2012, doi: 10.1109/MC.2012.57.

- [13] J. Park, J. Park, S. An, and S. Ryu, “JISSET: JavaScript IR-based Semantics Extraction Toolchain,” *Proceedings - 2020 35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020*, pp. 647–658, Sep. 2020, doi: 10.1145/3324884.3416632.
- [14] I. H. Madurapperuma, M. S. Shafana, and M. J. A. Sabani, “State-of-art frameworks for front-end and back-end web development,” *2nd International Conference on Science and Technology*, pp. 131–135, Aug. 2022.
- [15] A. Dattolo, M. Corbatto, and M. Angelini, “Authoring and Reviewing Bibliographies: Design and Development of a Visual Analytics Online Platform,” *IEEE Access*, vol. 10, pp. 21631–21645, 2022, doi: 10.1109/ACCESS.2022.3153027.
- [16] A. Paltoglou, V. E. Zafeiris, E. A. Giakoumakis, and N. A. Diamantidis, “Automated refactoring of client-side JavaScript code to ES6 modules,” *25th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2018 - Proceedings*, vol. 2018-March, pp. 402–412, Apr. 2018, doi: 10.1109/SANER.2018.8330227.
- [17] C. M. Kweon, “A Review of Heavy-Fueled Rotary Engine Combustion Technologies,” May 2011, doi: 10.21236/ADA545309.
- [18] A. Khandelwal, “Developing an Expense Tracking Application using React and Node.js,” 2022. [Online]. Available: <http://www.theseus.fi/handle/10024/785937>